

Pintos Project 3

Virtual Memory

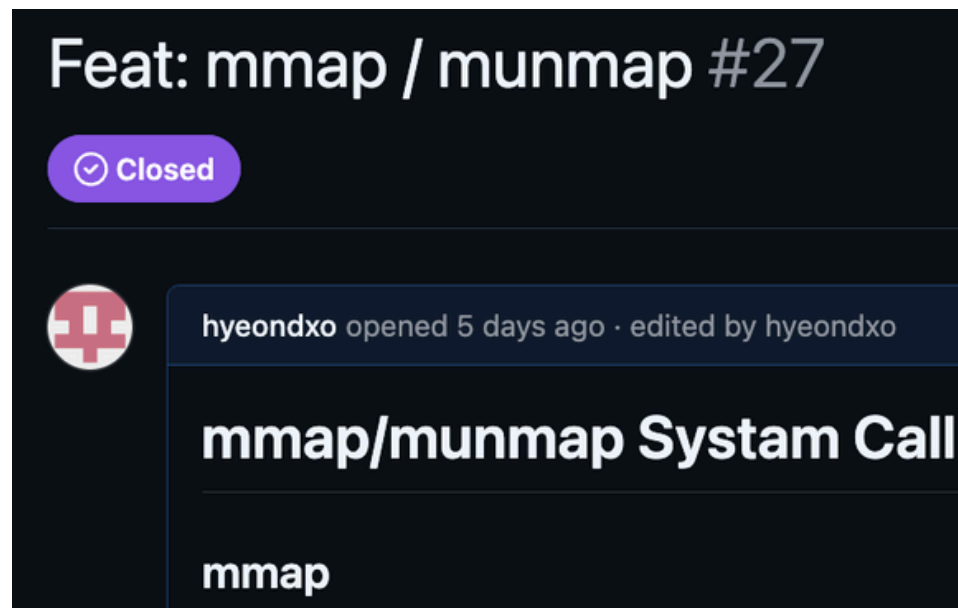
TEAM06 박도현, 진주영, 박시현

#1 기능 구현의 분담

매일 오전 10:30 데일리 스크럼 → 작업 계획 공유

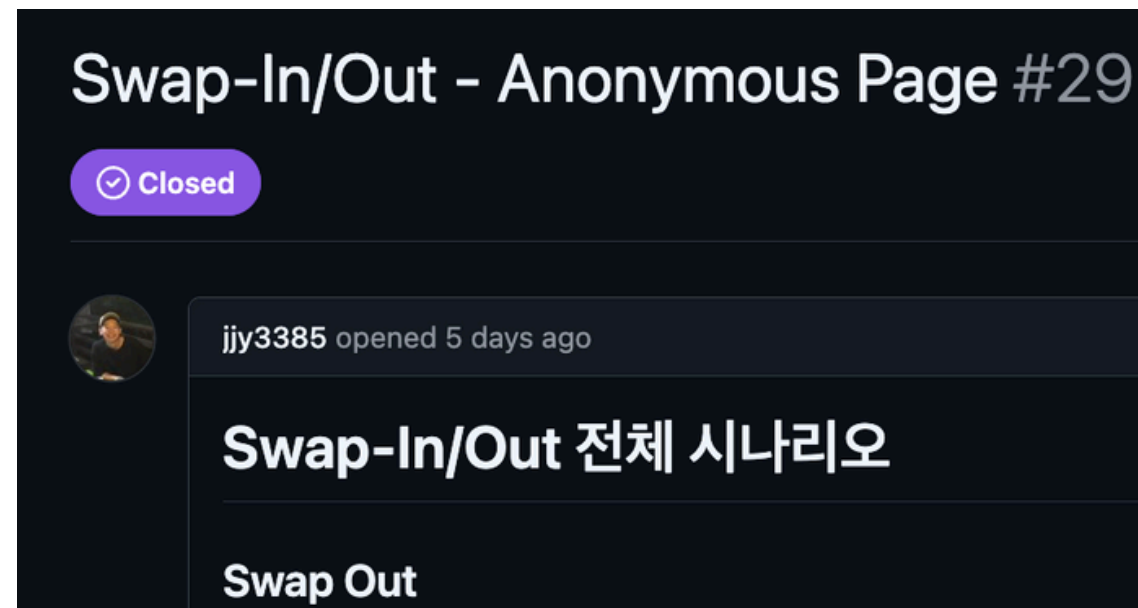
박도현

Stack Growth
Mmap/Munmap



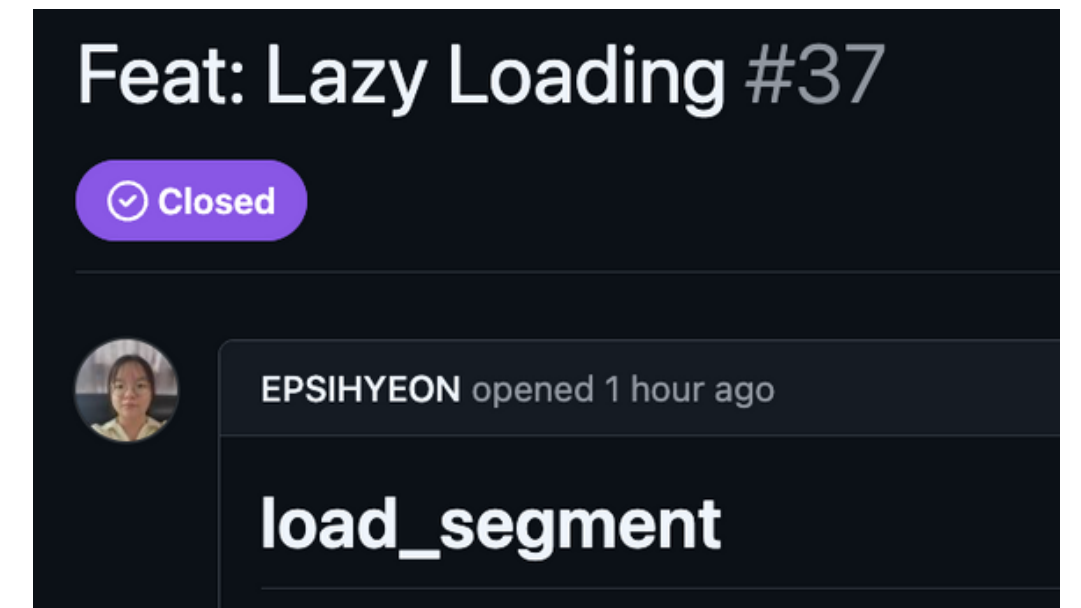
진주영

Frame Management
Swap In/Out – Anon



박시현

Lazy Loading
Swap In/Out – File



#1 기능 구현의 분담

매일 오전 10:30 데일리 스크럼 → 작업 계획 공유

박도현

Stack Growth
Mmap/Munmap

진주영

Frame Management
Swap In/Out – Anon

박시현

Lazy Loading
Swap In/Out – File

Feat: mmap / munmap #27

Closed



hyeondxo opened 5 days ago · edited by hyeondxo

mmap/munmap System Call

mmap

Swap-In/Out - Anonymous Page #29

Closed



jjy3385 opened 5 days ago

Swap-In/Out 전체 시나리오

Swap Out

Feat: Lazy Loading #37

Closed



EPSIHYEON opened 1 hour ago

load_segment

#2 Team Git Rules Branch

나만무 하는 것처럼 핀토스를 협업해보자!

git branch -r



6팀 브랜치 전략

- **main** : 최종 안정본 - 직접 push 금지
- **dev** : 통합 개발 브랜치 - 모든 PR의 base
- **feature/*** : 개인 기능 단위 작업 브랜치
 - 항상 dev에서 분기
- **fix/*** : 구현 기능에 대한 버그 수정 브랜치
 - 문제가 발생한 브랜치 혹은 dev에서 분기

(END)

```
origin/HEAD -> origin/main
origin/bugfix/kernel-panic
origin/dev
origin/feature/Lazy-Loading-For-Executable/anon
origin/feature/Memory-Mapped-File/munmap
origin/feature/Memory-Mapped-Files
origin/feature/Page-Cleanup
origin/feature/fix-Conflict
origin/feature/stack-growth
origin/fix/stack-growth
origin/main
upstream/dev
upstream/feature/Conflict-merge
upstream/feature/Conflit-swap
upstream/feature/File-Mapped-Page
upstream/feature/Implement-Supplemental-Page-Table
upstream/feature/Lazy-Loading-for-Executable/load
upstream/feature/Lazy-Loading-for-Executable/setup-stack
upstream/feature/Lazy-Loading-for-Executable/uninit
upstream/feature/Page-Cleanup
upstream/feature/Supplemental-Page-Table-Revisit
upstream/feature/Swap-in-out/Anonymous-Page
upstream/feature/fix-Conflict
upstream/main
```

#2 Team Git Rules Branch

나만무 하는 것처럼 핀토스를 협업해보자!

git branch -r



6팀 브랜치 전략

- **main** : 최종 안정본 - 직접 push 금지
- **dev** : 통합 개발 브랜치 - 모든 PR의 base
- **feature/*** : 개인 기능 단위 작업 브랜치
 - 항상 dev에서 분기
- **fix/*** : 구현 기능에 대한 버그 수정 브랜치
 - 문제가 발생한 브랜치 혹은 dev에서 분기

(END)

```
origin/HEAD -> origin/main
origin/bugfix/kernel-panic
origin/dev
origin/feature/Lazy-Loading-For-Executable/anon
origin/feature/Memory-Mapped-File/munmap
origin/feature/Memory-Mapped-Files
origin/feature/Page-Cleanup
origin/feature/fix-Conflict
origin/feature/stack-growth
origin/fix/stack-growth
origin/main
upstream/dev
upstream/feature/Conflict-merge
upstream/feature/Conflit-swap
upstream/feature/File-Mapped-Page
upstream/feature/Implement-Supplemental-Page-Table
upstream/feature/Lazy-Loading-for-Executable/load
upstream/feature/Lazy-Loading-for-Executable/setup-stack
upstream/feature/Lazy-Loading-for-Executable/uninit
upstream/feature/Page-Cleanup
upstream/feature/Supplemental-Page-Table-Revisit
upstream/feature/Swap-in-out/Anonymous-Page
upstream/feature/fix-Conflict
upstream/main
```

#3 Team Git Rules PR & Merge

안전한 병합과 버전 관리

PR & Merge 동선

1 작업

- `feature/*`
- push : `origin/feature/*`

2 Pull Request

- compare : `origin/feature/*`
- base : `upstream/dev`
- 코어타임 → 코드리뷰 후 merge

3 팀 동기화

- `git fetch upstream`
- `git rebase upstream/dev`

Team Repo PR history

-  Merge : Memory-Mapped-File/munmap, Conflict-merge (Test Passed: 135/140)
#32 by hyeondxo was merged yesterday
-  Merge: Memory-Mapped-Files, Swap-in-out/Anonymous-Page enhancement
#30 by hyeondxo was merged 2 days ago
-  Merge: feature/fix-Conflict, fix/stack-growth (Test Passed 108/140) enhancement
#23 by hyeondxo was merged 5 days ago
-  Feature/supplemental page table revisit enhancement
#22 by EPSIHYEON was merged 2 weeks ago
-  Feat: Memory Management, Anonymous Page (Test Passed: 61/141) enhancement
#20 by hyeondxo was merged 2 weeks ago • Approved
-  Fix: 모든 테스트에서 `exit(-1)`로 실패하는 문제 해결 bug
#19 by hyeondxo was merged 2 weeks ago
-  Merge: Lazy Loading for Executable enhancement
#18 by hyeondxo was merged 2 weeks ago

#3 Team Git Rules PR & Merge

안전한 병합과 버전 관리

PR & Merge 동선

1 작업

- `feature/*`
- push : `origin/feature/*`

2 Pull Request

- compare : `origin/feature/*`
- base : `upstream/dev`
- 코어타임 → 코드리뷰 후 merge

3 팀 동기화

- `git fetch upstream`
- `git rebase upstream/dev`

Team Repo PR history

	Merge : Memory-Mapped-File/munmap, Conflict-merge (Test Passed: 135/140)
	#32 by hyeondxo was merged yesterday
	Merge: Memory-Mapped-Files, Swap-in-out/Anonymous-Page enhancement
	#30 by hyeondxo was merged 2 days ago
	Merge: feature/fix-Conflict, fix/stack-growth (Test Passed 108/140) enhancement
	#23 by hyeondxo was merged 5 days ago
	Feature/supplemental page table revisit enhancement
	#22 by EPSIHYEON was merged 2 weeks ago
	Feat: Memory Management, Anonymous Page (Test Passed: 61/141) enhancement
	#20 by hyeondxo was merged 2 weeks ago • Approved
	Fix: 모든 테스트에서 exit(-1)로 실패하는 문제 해결 bug
	#19 by hyeondxo was merged 2 weeks ago
	Merge: Lazy Loading for Executable enhancement
	#18 by hyeondxo was merged 2 weeks ago

#4 저희가 협업에 얼마나 진심이었냐면요..

오프라인 코어타임이 불가능?

→ **온라인**으로라도 진행

구조체의 변경이나

커널 패닉이 발생하는 라인까지까지 공유

박시현(10기-10) 오후 3:23
<https://meet.google.com/qhp-mjwv-uzm>

 **meet.google.com**
Meet
Real-time meetings by Google. Using your browser, share your video, desktop, and presentations with teammates and customers.
👍 1 👏 1 🗨️ 1 😊
7시 30분에 구글 미트에서 뵙겠습니다!

박도현(10기-34) 오후 5:15
요거때매 출력차이로 실패하던게 몇 개 있었네요 로그가 이상해서 발견했는데 주석처리하니까 되는게 많아요

image.png ▼

```
if (user) {  
    // printf("heeelllello"); ?? 이거때매 테스트 평평  
    exit(-1);  
}
```

exception.c 맨 아래쪽입니다..!

박시현(10기-10) 오후 5:57
방금 fixconflict에 63개(userprog 다 통과한것 올렸습니다!!!

박도현(10기-34) 오후 5:58
혹시
총 몇 개 통과되셨나요??

박시현(10기-10) 오후 6:09
정확히는 모르겠네요.. 나중에 다 돌려보겠습니다!! 주영님 말씀으로는 50개정도 통과 못하셨다고 하네요

박도현(10기-34) 오후 6:25
고생하셨어요! 저도 stack growth 문제 해결하고 관련 테스트 통과했는데 한 번 머지해보겠습니다!

박시현(10기-10) 오전 10:47

```
enum page_status {  
    IN_FRAME, //페이지가 물리 프레임안에 존재  
    IN_FILE, //페이지가 파일 시스템 안에 존재  
    IN_SWAP //페이지가 스왑디스크 안에 존재  
};  
  
page 구조체에  
enum page_status status;  
추가  
사실 필요할지는 모르겠으나.. 일단 추가했습니다! 또 수정되면 알려드리겠습니다 !
```

진주영(정글10기-31) 오후 7:17
시현님 feature/Swap-in-out/Anonymous-Page 브랜치로 팀레포에 올렸습니다

make check 은 이전이랑 똑같고... swap_anon 테스트케이스 돌리면 vm.c 212 줄에서 커널패닉 발생합니다.
static struct frame *vm_get_frame(void) 함수 malloc() 내부입니다..π

swap out 하면서 뭔가 free 를 잘못된거 같은데 아직은 모르겠네요

#4 저희가 협업에 얼마나 진심이었나면요..

오프라인 코어타임이 불가능?

→ 온라인으로도 진행

구조체의 변경이나

커널 패닉이 발생하는 라인까지까지 공유

박시현(10기-10) 오후 3:23
<https://meet.google.com/qhp-mjwv-uzm>

meet.google.com

Meet

Real-time meetings by Google. Using your browser, share your video, desktop, and presentations with teammates and customers.

👍 1 👏 1 🗨️ 1 😊 1

7시 30분에 구글 미트에서 뵙겠습니다!

박도현(10기-34) 오후 5:15
요거때매 출력차이로 실패하던게 몇 개 있었네요 로그가 이상해서 발견했는데 주석처리하니까 되는게 많아요

image.png ▼

```
if (user) {  
    // printf("heeeello"); ?? 이거때매 테스트 평평  
    exit(-1);  
}
```

exception.c 맨 아래쪽입니다..!

박시현(10기-10) 오후 5:57
방금 fixconflict에 63개(userprog 다 통과한것 올렸습니다!!!

박도현(10기-34) 오후 5:58
혹시

총 몇 개 통과되셨나요??

박시현(10기-10) 오후 6:09
정확히는 모르겠네요.. 나중에 다 돌려보겠습니다!! 주영님 말씀으로는 50개정도 통과 못하셨다고 하네요

박도현(10기-34) 오후 6:25
고생하셨어요! 저도 stack growth 문제 해결하고 관련 테스트 통과했는데 한 번 머지해보겠습니다!

박시현(10기-10) 오전 10:47

```
enum page_status {  
    IN_FRAME, //페이지가 물리 프레임안에 존재  
    IN_FILE, //페이지가 파일 시스템 안에 존재  
    IN_SWAP //페이지가 스왑디스크 안에 존재  
};  
  
page 구조체에  
enum page_status status;  
추가  
사실 필요할지는 모르겠으나.. 일단 추가했습니다! 또 수정되면 알려드리겠습니다!
```

박시현(10기-10) 오후 7:18

시현님 feature/Swap-in-out/Anonymous-Page 브랜치로 팀레포에 올렸습니다

make check 은 이전이랑 똑같고... swap_anon 테스트케이스 돌리면 vm.c 212 줄에서 커널패닉 발생합니다.
static struct frame *vm_get_frame(void) 함수 malloc() 내부입니다..ㅠ

swap out 하면서 뭔가 free 를 잘못된거 같은데 아직은 모르겠네요

#5 모두가 함께 이루어낸 ALL PASS

dev→main으로의 **마지막 PR**

Feat: PintOS VM 최종본 (Test Passed: 140/140) #35
 Merged EPSIHYEON merged 4 commits into `main` from `dev` 18 hours ago

Conversation 2

Commits 4

Checks 0

Files changed 13

**EPSIHYEON** commented 18 hours ago
COW 제외 올패스
 1

 **jjy3385** approved these changes 18 hours ago

jjy3385 left a comment
최고!
 2

 **hyeondxo** approved these changes 18 hours ago

hyeondxo left a comment
3주간 고생 많으셨습니다!!
 2

test 140/140 finish

=== Test Summary ===
Passed: 140

SWAP-ANON 테스트 분석

WITH. 메모리 영역

진주영

#1 SWAP-ANON.C 테스트코드에서 메모리 영역 예측

```
#define PAGE_SHIFT 12
#define PAGE_SIZE (1 << PAGE_SHIFT)
#define ONE_MB (1 << 20) // 1MB
#define CHUNK_SIZE (20 * ONE_MB)
#define PAGE_COUNT (CHUNK_SIZE / PAGE_SIZE)

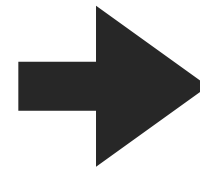
static char big_chunks[CHUNK_SIZE];

void test_main(void) {
    size_t i;
    void* pa;
    char* mem;

    for (i = 0; i < PAGE_COUNT; i++) {
        if (!(i % 512)) msg("write sparsely over page %zu", i);
        mem = (big_chunks + (i * PAGE_SIZE));
        *mem = (char)i;
    }

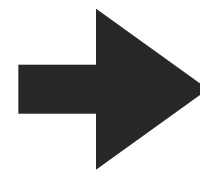
    for (i = 0; i < PAGE_COUNT; i++) {
        mem = (big_chunks + (i * PAGE_SIZE));
        if ((char)i != *mem) {
            fail("data is inconsistent");
        }
        if (!(i % 512)) msg("check consistency in page %zu", i);
    }
}
```

1. static char big_chunks[CHUNK_SIZE]



20MB 크기의 배열
초기화 되지 않은 전역변수
BSS 영역

2. void test_main(void)



메인 함수
함수의 기계어 코드
코드 영역

#1 SWAP-ANON.C 테스트코드에서 메모리 영역 예측

```
#define PAGE_SHIFT 12
#define PAGE_SIZE (1 << PAGE_SHIFT)
#define ONE_MB (1 << 20) // 1MB
#define CHUNK_SIZE (20 * ONE_MB)
#define PAGE_COUNT (CHUNK_SIZE / PAGE_SIZE)

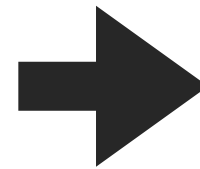
static char big_chunks[CHUNK_SIZE];

void test_main(void) {
    size_t i;
    void* pa;
    char* mem;

    for (i = 0; i < PAGE_COUNT; i++) {
        if (!(i % 512)) msg("write sparsely over page %zu", i);
        mem = (big_chunks + (i * PAGE_SIZE));
        *mem = (char)i;
    }

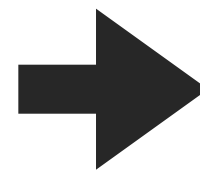
    for (i = 0; i < PAGE_COUNT; i++) {
        mem = (big_chunks + (i * PAGE_SIZE));
        if ((char)i != *mem) {
            fail("data is inconsistent");
        }
        if (!(i % 512)) msg("check consistency in page %zu", i);
    }
}
```

3. "write sparsely over ~" 같은 문자열 리터럴



상수 문자열 데이터
읽기 전용 데이터
RODATA 영역

4. size_t i; char *mem; 같은 지역변수



메인 함수 내에서 사용되는 지역변수
함수 호출 시 생성/종료 시 반환
스택 영역

#2 SWAP-ANON.C 실행 시 흐름

load_segment() 와 lazy_load_segment()

```
/* 파일에서 읽어와야 할 내용이 있는 세그먼트 (code, rodata, data 등) */
if (page_read_bytes > 0) {
    aux->file = file_reopen(file);
    aux->ofs = ofs;
} else {
    /* 파일 내용 없이 0으로만 채워야 하는 세그먼트 (주로 BSS) */
    aux->file = NULL;
    aux->ofs = 0;
}
aux->read_bytes = page_read_bytes;
aux->zero_bytes = page_zero_bytes;
aux->mapping = NULL;

if (!vm_alloc_page_with_initializer(VM_ANON, upage, writable, lazy_load_segment, aux)) {
    free(aux);
    return false;
}
```

파일에서 읽어와야할 내용이 있음
→ **code, rodata, data 영역**

파일 내용없이 0으로만 채워야 함
→ **bss 영역**

```
/* 파일에서 읽어와야 할 내용이 있는 세그먼트 (code, rodata, data 등) */
if (file != NULL) {
    if (file_read_at(file, kpage, page_read_bytes, ofs) != (int)page_read_bytes) {
        free(aux_data);
        return false; // 파일 읽기 실패는 false 반환
    }
}
/* code, rodata, data 등 : 나머지 영역을 0으로 채움 */
/* BSS : 페이지 전체를 0으로 채움 */
memset(kpage + page_read_bytes, 0, page_zero_bytes);
```

#2 SWAP-ANON.C 실행 시 흐름

첫번째 루프 실행

```
#define PAGE_SHIFT 12
#define PAGE_SIZE (1 << PAGE_SHIFT)
#define ONE_MB (1 << 20) // 1MB
#define CHUNK_SIZE (20 * ONE_MB)
#define PAGE_COUNT (CHUNK_SIZE / PAGE_SIZE)

static char big_chunks[CHUNK_SIZE];

void test_main(void) {
    size_t i;
    void* pa;
    char* mem;

    for (i = 0; i < PAGE_COUNT; i++) {
        if (!(i % 512)) msg("write sparsely over page %zu", i);
        mem = (big_chunks + (i * PAGE_SIZE));
        *mem = (char)i;
    }

    for (i = 0; i < PAGE_COUNT; i++) {
        mem = (big_chunks + (i * PAGE_SIZE));
        if ((char)i != *mem) {
            fail("data is inconsistent");
        }
        if (!(i % 512)) msg("check consistency in page %zu", i);
    }
}
```

SPT 에서 페이지 선택하여 물리주소 매핑(예시)

가상주소	타입	물리주소	섹션	내용
0x4747f000	ANON	0x8004521000	스택	test_main() 지역변수의 값(i,*pa,*mem)
0x805000	ANON		BSS	모두 0
0x806000	ANON		BSS	모두 0
...				
0x8bb000	ANON		BSS	모두 0

#2 SWAP-ANON.C 실행 시 흐름

첫번째 루프 실행

```
#define PAGE_SHIFT 12
#define PAGE_SIZE (1 << PAGE_SHIFT)
#define ONE_MB (1 << 20) // 1MB
#define CHUNK_SIZE (20 * ONE_MB)
#define PAGE_COUNT (CHUNK_SIZE / PAGE_SIZE)

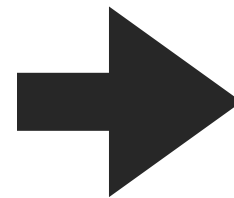
static char big_chunks[CHUNK_SIZE];

void test_main(void) {
    size_t i;
    void* pa;
    char* mem;

    for (i = 0; i < PAGE_COUNT; i++) {
        if (!(i % 512)) msg("write sparsely over page %zu", i);
        mem = (big_chunks + (i * PAGE_SIZE));
        *mem = (char)i;
    }

    for (i = 0; i < PAGE_COUNT; i++) {
        mem = (big_chunks + (i * PAGE_SIZE));
        if ((char)i != *mem) {
            fail("data is inconsistent");
        }
        if (!(i % 512)) msg("check consistency in page %zu", i);
    }
}
```

mem = (big_chunks + (i * PAGE_SIZE));
*mem = (char)i;



페이지 단위로 배열 접근
페이지 폴트 발생!

#2 SWAP-ANON.C 실행 시 흐름

스와핑 발생

```
/* 새로운 물리 프레임을 얻을 때 사용 */
static struct frame *vm_get_frame(void) {
    struct frame *frame;
    void *kva = palloc_get_page(PAL_USER);
    /* 할당 실패 시 기존 프레임 재활용 */
    if (!kva) {
        frame = vm_evict_frame();
        /* 프레임 초기화 */
        frame->page = NULL;
        return frame;
    }
}
```

페이지 SWAP-OUT

가상주소	타입	물리주소	섹션	내용
0x4747f000	ANON	0x8004521000	스택	test_main() 지역변수의 값(i,*pa,*mem)
0x805000	ANON	0x8004929000	BSS	512, 나머지는 모두 0
0x806000	ANON	0x800492a000	BSS	513, 나머지는 모두 0
...				
0x8bb000	ANON	0x80049df000	BSS	1206, 나머지는 모두 0

#2 SWAP-ANON.C 실행 시 흐름

스와핑 발생

```
/* 새로운 물리 프레임을 얻을 때 사용 */
static struct frame *vm_get_frame(void) {
    struct frame *frame;
    void *kva = pallocc_get_page(PAL_USER);
    /* 할당 실패 시 기존 프레임 재활용 */
    if (!kva) {
        frame = vm_evict_frame();
        /* 프레임 초기화 */
        frame->page = NULL;
        return frame;
    }
}
```

페이지 SWAP-IN & SWAP-OUT

가상주소	타입	물리주소	섹션	내용
0x4747f000	ANON	0x8004521000	스택	test_main() 지역변수의 값(i,*pa,*mem)
0x8bc000	ANON	0x8004929000	BSS	1207, 나머지는 모두 0
0x806000	ANON	0x800492a000	BSS	513, 나머지는 모두 0
...				
0x8bb000	ANON	0x80049df000	BSS	1206, 나머지는 모두 0

#2 SWAP-ANON.C 실행 시 흐름

스와핑 발생

```
/* 새로운 물리 프레임을 얻을 때 사용 */
static struct frame *vm_get_frame(void) {
    struct frame *frame;
    void *kva = palloc_get_page(PAL_USER);
    /* 할당 실패 시 기존 프레임 재활용 */
    if (!kva) {
        frame = vm_evict_frame();
        /* 프레임 초기화 */
        frame->page = NULL;
        return frame;
    }
}
```

페이지 SWAP-IN & SWAP-OUT

가상주소	타입	물리주소	섹션	내용
0x4747f000	ANON	0x8004521000	스택	test_main() 지역변수의 값(i,*pa,*mem)
0x8bc000	ANON	0x8004929000	BSS	1207, 나머지는 모두 0
0x8bd000	ANON	0x800492a000	BSS	1208, 나머지는 모두 0
...				
0x8bb000	ANON	0x80049df000	BSS	1206, 나머지는 모두 0

트러블슈팅 _load 관련

박시현

multi-recurse

```
Executing 'multi-recurse 15':
```

```
(multi-recurse) begin 15
```

```
(multi-recurse) exec("multi-recurse 14")
```

```
(multi-recurse) begin 14
```

```
(multi-recurse) exec("multi-recurse 13")
```

```
(multi-recurse) begin 13
```

```
(multi-recurse) exec("multi-recurse 12")
```

```
Kernel PANIC at ../../threads/thread.c:259 in thread_current(): assertion `is_thread(t)' failed.
```

됐다가...
안 됐다가....



process.c의 load

```
// parsing
char *args[32];
args[0] = program_command;
char *target;
int num = 1;

while ((target = strtok_r(NULL, " ", &str_point)) != NULL) {
    if (num >= 32) { // args 배열이 꽉 찼으면 더 이상 받지 않음
        break;
    }
    args[num] = target;
    num++;
}
const int argc = num;
char *args_address[32];
```

배열 크기

64- > 32로 변경

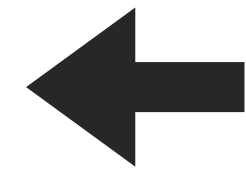
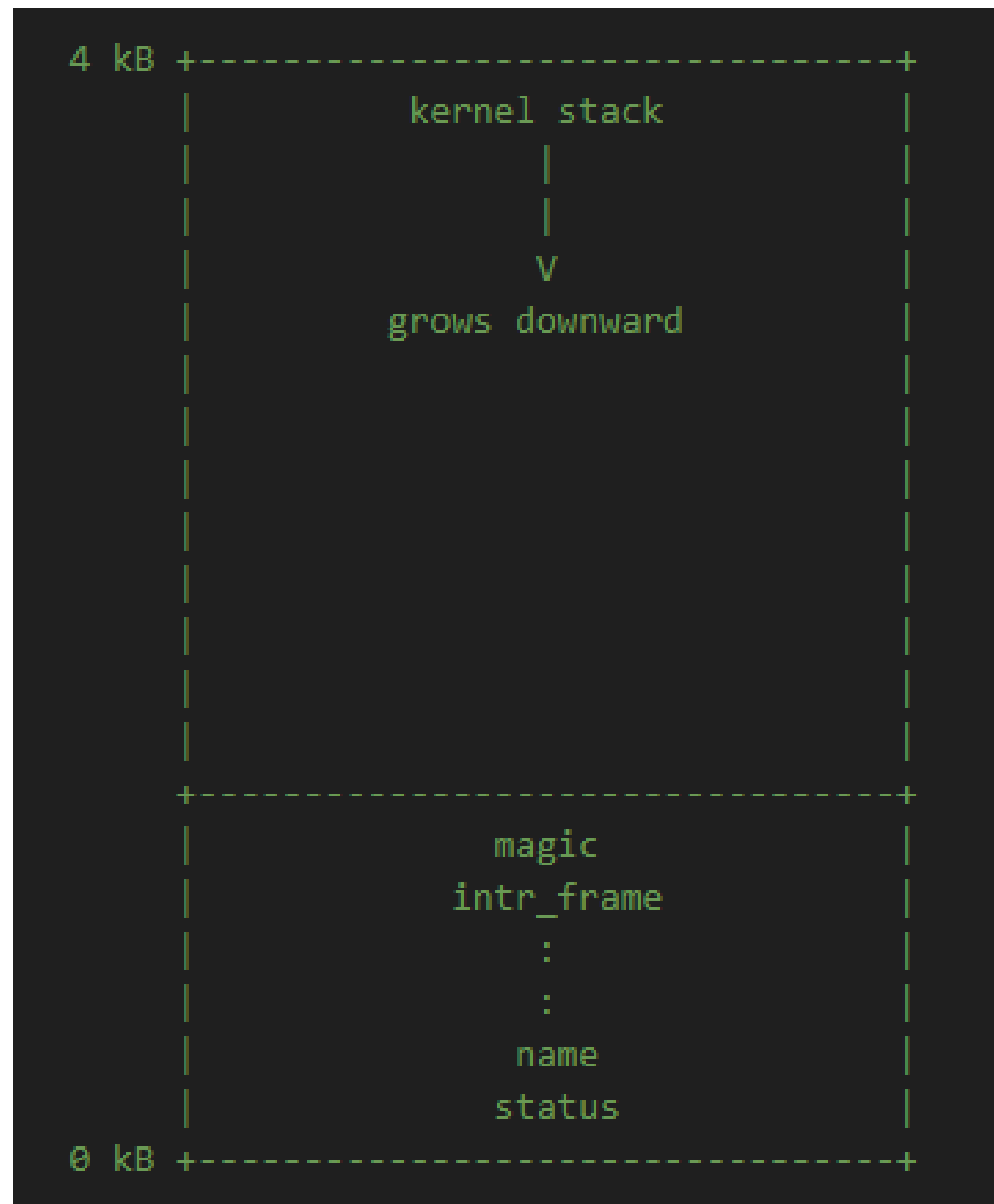
```
FAIL tests/vm/cow/cow-simple  
1 of 141 tests failed.
```

성공!

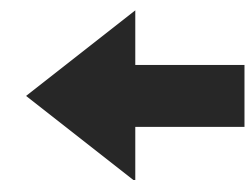


왜일까?

쓰레드 페이지 구조



커널 스택



쓰레드 구조체
(struct thread)

Thread.h

쓰레드의 커널 스택에 들어가는 부분

load에서 쓰는
지역 변수들이
들어감

```
struct thread *t = thread_current();
struct ELF ehdr;
struct file *file = NULL;
off_t file_ofs;
bool success = false;
int i;

char *program_command;
char *str_point;
```

```
// parsing
char *args[32];
args[0] = program_command;
char *target;
int num = 1;

while ((target = strtok_r(NULL, " ", &str_point)) != NULL) {
    if (num >= 32) { // args 배열이 꽉 찼으면 더 이상 받지 않음
        break;
    }
    args[num] = target;
    num++;
}
const int argc = num;
char *args_address[32];
```

process.c의 load

하나의 쓰레드 페이지에 들어가는 요소는?

쓰레드



**struct thread에 선언한 내용의 전체 용량
+ load의 지역 변수 들 + 그외**

<

4KB
(4096 bytes,
한 페이지)

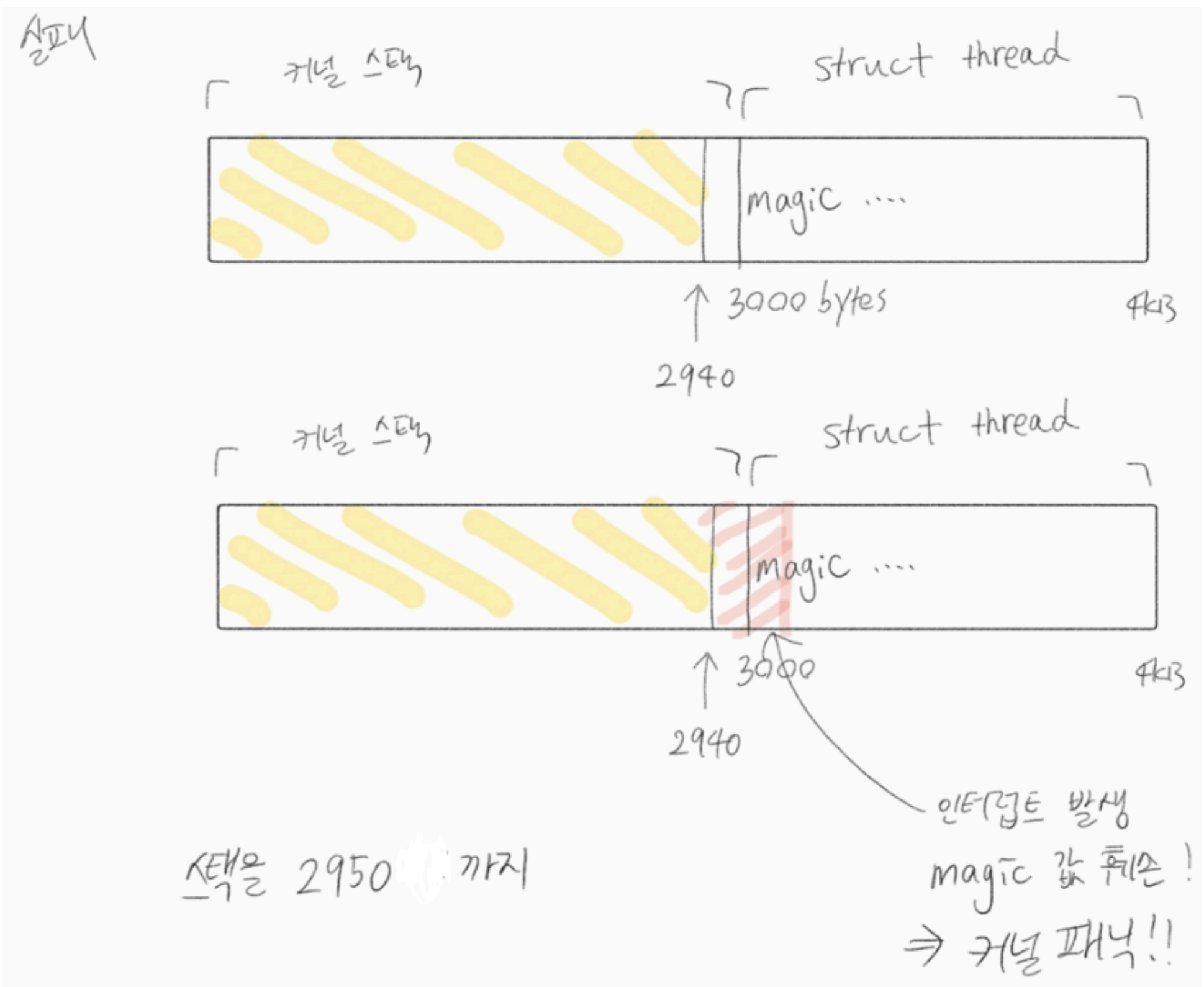
왜 배열 크기가 64였을때는 됐다가 안됐다가 했을까?

실패의 경우:

2940bytes + 커널 스택
인터럽트 크기 > 최대치
(3040 bytes) (3000 bytes)

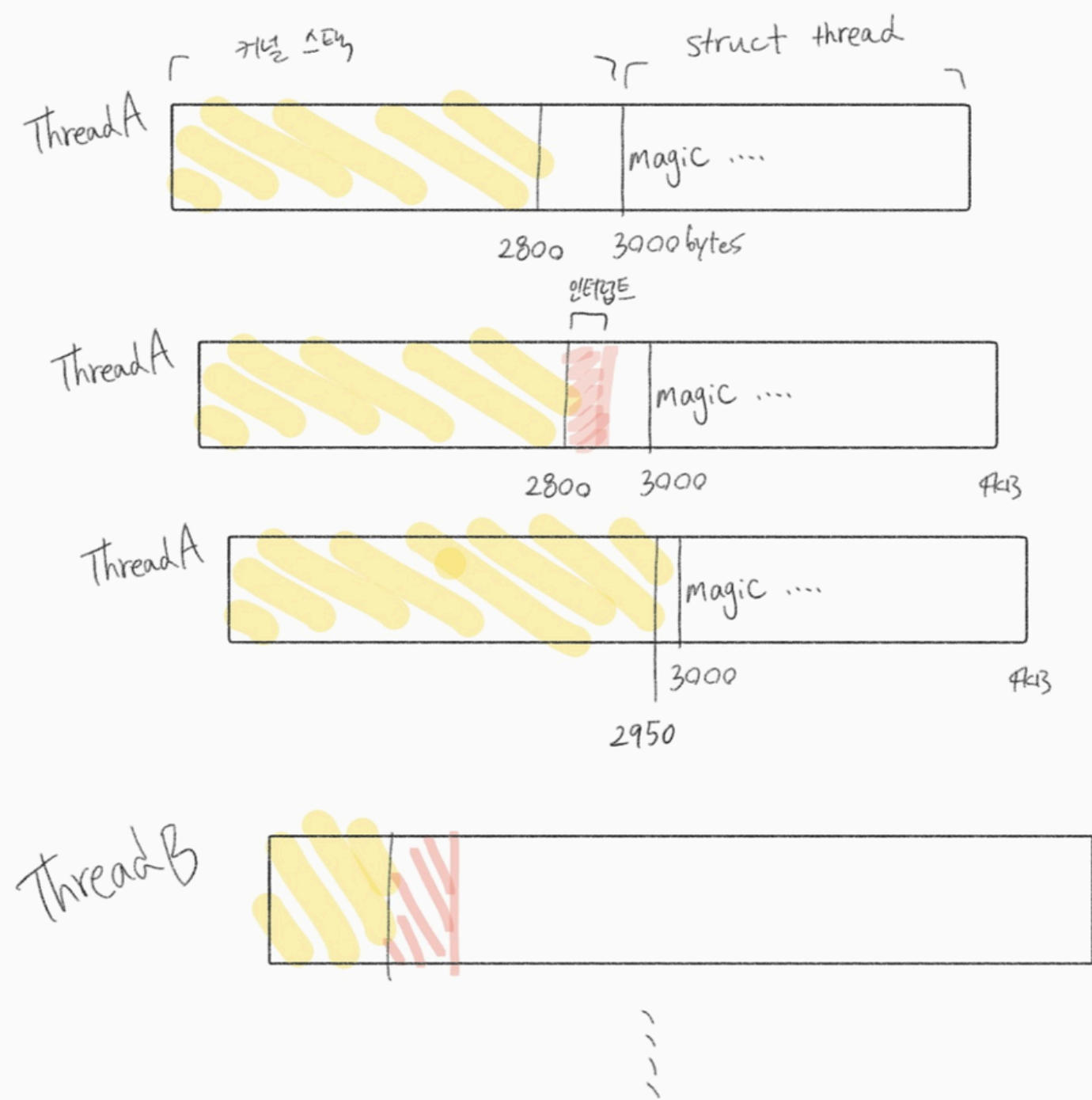
커널 패닉!!!

2950bytes 까지 커널 스택에
데이터를 넣어야함



왜 배열 크기가 64였을때는 됐다가 안됐다가 했을까?

성공의 경우:



2800bytes +
인터럽트 크기 < 커널 스택
(2900 bytes) 최대치
(3000 bytes)

성공

2950bytes 까지 커널 스택에
데이터를 넣어야함

왜 같은 조건인데도 다른 결과가 나왔을까?

인터럽트 자체는 주기적으로 일어나지만

커널 스택이 데이터를 쌓는 속도는 일정하지 않기 때문!

배열 크기를 64→ 32로 바꾸면?



**데이터를 덜 쌓아도 되므로
인터럽트가 언제 와도
상관없음!**

안정적으로 성공!

THANK YOU

Virtual Memory

TEAM06 박도현, 진주영, 박시현